

エラトステネスのふるい

アルゴリズムの世界で、特定の数までの **素数（1とその数自身でしか割り切れない数）** をすべて効率よく見つけるための手法が、**エラトステネスのふるい** です。

紀元前3世紀のギリシャの数学者エラトステネスが考案したもので、「ふるいにかけて、素数でないものを落としていく」という直感的な仕組みになっています。

仕組み（ステップ）

例えば、1から30までの素数を見つける手順は以下の通りです。

1. **準備**：2から30までの数字を書いた表を用意します（1は素数ではないので除外）。

1	2	3	4	5	6
7	8	9	10	11	12
13	14	15	16	17	18
19	20	21	22	23	24
25	26	27	28	29	30

2. **2を選ぶ**：表に残っている最小の数「2」は素数です。次に、**2の倍数（4, 6, 8...）** をすべて消します。

1	2	3	4	5	6
7	8	9	10	11	12
13	14	15	16	17	18
19	20	21	22	23	24
25	26	27	28	29	30

3. **3を選ぶ**：次に残っている最小の数「3」は素数です。次に、**3の倍数（6, 9, 12...）** をすべて消します。

1	2	3	4	5	6
7	8	9	10	11	12
13	14	15	16	17	18
19	20	21	22	23	24
25	26	27	28	29	30

4. **繰り返し**：次に残っている数（次は5）について同様の操作を繰り返します。

1	2	3	4	5	6
7	8	9	10	11	12
13	14	15	16	17	18
19	20	21	22	23	24
25	26	27	28	29	30

5. **終了**：30の平方根（5.47722557505）まで調べれば、残った数字はすべて素数です。

1	2	3	4	5	6
7	8	9	10	11	12
13	14	15	16	17	18
19	20	21	22	23	24
25	26	27	28	29	30

なぜ「割り算」をしないのか？（メリット）

ある数が素数かどうかを判定する際、通常は「2, 3, 4...と順番に割ってみる」という方法をとります。しかし、数が多い場合、この「試し割り」は非常に時間がかかります。

エラトステネスのふるいの凄さは、**「割り算をせず、倍数を消していく（足し算）だけで済む」**点にあります。これにより、大量の素数を一気に、かつ高速に求めることができます。

計算量と効率化のポイント

このアルゴリズムの計算量は ($O(n \log \log n)$) とされており、ほぼ (n) に比例する非常に優れた効率を誇ります。

さらに効率を上げるテクニックとして、以下のようなものがあります。

- **平方根までで止める**：までの素数を探すなら、までの倍数を消せば十分です（それ以降の合成数はすでに消されているため）。
- **奇数だけを対象にする**：2以外の偶数は最初から除外して計算すれば、メモリと時間を半分に節約できます。

プログラムでの実装（疑似言語）

試験で出題される際は、配列を「旗（フラグ）」に見立てて、素数でない場所を **false** に書き換えていく処理が一般的です。

```
/* エラトステネスのふるい（n までの素数を判定する処理） */
整数型: n ← 30

/* 1から始まる配列を定義し、全要素を true で初期化 */
/* 添字 1 から n までの n要素の配列を用意する */
論理型[]: isPrime ← [n要素のすべての値を true に初期化]

/* 1 は素数ではないので false にする（0 は1スタートなので存在しない） */
isPrime[1] ← false

/* ループ変数の上限となる √n を計算しておく（※試験での平方根表現） */
整数型: limit ← 平方根(n)

/* 外側のループ: 2 から √n まで */
for (整数型: i を 2 から limit まで 1 ずつ増やす)
  if (isPrime[i] = true)
    /* 内側のループ: iの2乗からnまで、i ずつ増やす */
    for (整数型: j を i × i から n まで i ずつ増やす)
      isPrime[j] ← false /* i の倍数をふるい落とす */
    endfor
  endif
endfor
```

実社会での活用

素数は現代のコンピュータセキュリティ、特に **公開鍵暗号（RSA暗号など）** の根幹を支えています。巨大な素数を見つける技術は、データの安全性を守るために非常に重要です。
